Operating Systems Assignment 2 – Task 5 (Q5)

Synchronisation: Critical Sections, Semaphores and Monitors – Railway Ticket Booking (CO2, K5 – Evaluate, 11 marks). Environment: Ubuntu 24.04.5 LTS, gcc 13.3.0, 2 CPUs. Compile: `gcc -std=c11 -O1 -Wall -Wextra -pthread`. Structure follows the expected flow *Model* → *Design* → *Implement* → *Measure* → *Compare* → *Evaluate* → *Recommend*.

1. Model: the problem

Thousands of agents book seats at once. The seat map is shared data. Two agents can both read “seat 12 is free” and both sell it (a **race condition**) unless the check-and-update is a **critical section**. A single global lock removes the race but serialises every booking.

2. Design: from the principle to a solution

Principle given: a critical-section solution must guarantee (1) **mutual exclusion** – at most one agent updates a seat at a time; (2) **progress** – if nobody is in the critical section, an agent that wants in is not blocked by agents outside it, and the decision cannot be postponed forever; (3) **bounded waiting** – a limit on how many others can enter before a waiting agent. My design turns these into rules:

Requirement	Design rule for the booking system
Mutual exclusion	Check-free-then-mark-sold on a seat is one atomic section guarded by a lock that covers that seat.
Progress	Locks are held only inside the section (never while waiting for the customer); agents booking different seats never block each other.
Bounded waiting	Use fair/blocking primitives (mutex or semaphore queue); no agent can be overtaken indefinitely, and critical sections are short.
Performance	Use the finest lock granularity whose bookkeeping is still simple (Section 8).

3. Peterson's solution: why it works for two processes, and why it may fail today

```
shared: flag[2] = {false,false}; turn
Pi:  flag[i] = true;  turn = j;
     while (flag[j] && turn == j) ;      // busy wait
     ... critical section ...
     flag[i] = false;
```

Requirement	Argument
Mutual exclusion	Pi enters only if flag[j]==false or turn==i. If both were inside, both flags are true, so turn==i and turn==j would both have to hold, which is impossible (turn is a single variable and each process sets it to the *other's* id before waiting). So the one that wrote turn last waits.
Progress	If Pj is not interested, flag[j]==false and Pi enters at once. If both want in, turn decides within one step.
Bounded waiting	After leaving, Pj sets flag[j]=false; if it comes back it sets turn=i, giving Pi priority, so Pi waits for at most one entry of Pj.

Why it fails on modern processors: the proof assumes loads and stores happen in program order. Modern CPUs and compilers reorder them (store buffers let the load of `flag[j]` complete before the store to `flag[i]` is visible to the other core), so both threads may see the other's flag as false and both enter. Measured (`peterson.c`, 2 threads × 2,000,000 increments):

```
plain  expected 4000000  got 3988839  MUTUAL EXCLUSION VIOLATED
plain  expected 4000000  got 3993753  MUTUAL EXCLUSION VIOLATED
plain  expected 4000000  got 3984337  MUTUAL EXCLUSION VIOLATED
fenced expected 4000000  got 4000000  mutual exclusion held
fenced expected 4000000  got 4000000  mutual exclusion held
fenced expected 4000000  got 4000000  mutual exclusion held
```

With ordinary variables roughly 6,000–16,000 of 4,000,000 updates were lost in each run (mutual exclusion violated). With sequentially consistent atomics (which insert memory fences) the result is exact. In practice use mutexes or atomics rather than Peterson; also busy-waiting wastes CPU.

4. Semaphores protecting the seat counter

```
wait(S):    S = S - 1;  if (S < 0) block the caller on S's queue     (atomic)
signal(S):  S = S + 1;  if (S <= 0) wake one blocked process           (atomic)

Counting semaphore  seats = number of free seats (e.g. 40):
    book():    wait(seats);     // blocks when 0 seats are left (or the booking is refused with try_wait)
    cancel():  signal(seats);

Binary semaphore  mutex = 1  (protects the seat map / counter update):
    wait(mutex);  mark seat sold; sold++;  signal(mutex);
```

The counting semaphore keeps the number of sold seats ≤ capacity (a resource count); the binary semaphore gives mutual exclusion over the data structure. Blocked processes sleep in the semaphore's queue (no CPU), and a FIFO queue gives bounded waiting.

5. Bounded buffer, readers–writers, dining philosophers with semaphores

5.1 Bounded buffer (producer–consumer)

```
semaphore empty = N, full = 0, mutex = 1
producer: wait(empty); wait(mutex); buf[in]=item; in=(in+1)%N; signal(mutex); signal(full)
consumer: wait(full);  wait(mutex); item=buf[out]; out=(out+1)%N; signal(mutex); signal(empty)
```

empty blocks producers when the buffer is full, **full** blocks consumers when it is empty, and **mutex** protects the in/out indices when there are several producers or consumers. Order matters: wait on the counting semaphore *before* the mutex, otherwise a producer holding the mutex while waiting for a free slot deadlocks the consumers.

5.2 Readers–writers (readers' priority)

```
semaphore rw = 1, mutex = 1;  int readcount = 0
reader: wait(mutex); readcount++; if (readcount==1) wait(rw); signal(mutex);
        ... read ...
        wait(mutex); readcount--; if (readcount==0) signal(rw); signal(mutex)
writer: wait(rw);  ... write ...;  signal(rw)
```

Many readers (agents just viewing seat availability) share access; a writer (booking) gets exclusive access. Weakness: a continuous stream of readers can starve writers; the fix is a writers'-priority or fair (turnstile) variant.

5.3 Dining philosophers

Five philosophers, one semaphore per fork. If everyone picks up the left fork then the right, all hold one fork and wait forever (deadlock, circular wait). Fix used here: resource ordering – pick up the lower-numbered fork first, which breaks the circular wait (other fixes: allow at most 4 at the table; pick both forks inside a critical section). Program `dining.c`:

```
ordered: all philosophers finished, meals = 200 200 200 200 200
ordered exit=0
naive exit=124 (124 = hung: deadlock, killed by timeout)
```

The naive version hangs (killed by timeout after 5 s); the ordered version completes all 200 meals for each philosopher.

6. Monitors and condition variables

A monitor bundles the shared data, the lock and the waiting conditions in one construct: only one thread is active inside at a time, and `wait(cond)` releases the monitor lock and sleeps atomically; `signal(cond)` wakes a waiter. This is easier to get right than raw semaphores because there is no separate empty/full/mutex ordering to get wrong; the waiting condition is written in plain code (`while (count==N) wait(not_full)`).

```
put(x):  lock(m); while (count == N) cond_wait(not_full, m);
         buf[in]=x; in=(in+1)%N; count++; cond_signal(not_empty); unlock(m)
get():   lock(m); while (count == 0) cond_wait(not_empty, m);
         x=buf[out]; out=(out+1)%N; count--; cond_signal(not_full); unlock(m); return x
```

Implemented in `monitor.c` with pthread mutex + condition variables. 4 producers + 4 consumers, 100 runs:

```
monitor 4x4: 100/100 correct
```

7. Test the design: bounded buffer program (Complete the Code, TODO 1 and TODO 2)

TODO 1 (producer): `sem_wait(&empty)`, lock mutex, insert, unlock, `sem_post(&full)`. TODO 2 (consumer): `sem_wait(&full)`, lock mutex, remove, unlock, `sem_post(&empty)`. Items 1..20 are produced, so the sum must be 210 and every item must be consumed exactly once (checked with a per-item counter).

```
/* Bounded buffer (producer-consumer) with counting semaphores + mutex.
   Items 1..20 are produced; the sum of consumed items must be 210 and every item consumed exactly once.
   gcc -std=c11 -O1 -Wall -Wextra -pthread -o bb bb.c
   ./bb &lt;producers&gt; &lt;consumers&gt; [nomutex]     (producers and consumers must divide 20) */
#define _GNU_SOURCE
#include &lt;stdio.h&gt;
#include &lt;stdlib.h&gt;
#include &lt;string.h&gt;
#include &lt;pthread.h&gt;
#include &lt;semaphore.h&gt;
#include &lt;stdatomic.h&gt;
#include &lt;sched.h&gt;
#define BUF 5
#define ITEMS 20
static int buf[BUF], in = 0, out = 0, P, C, use_mutex = 1;
static sem_t empty_s, full_s;
static pthread_mutex_t m = PTHREAD_MUTEX_INITIALIZER;
static atomic_long sum; static atomic_int seen[ITEMS + 1];
#define LOCK()   do { if (use_mutex) pthread_mutex_lock(&amp;m); } while (0)
#define UNLOCK() do { if (use_mutex) pthread_mutex_unlock(&amp;m); } while (0)
static void *producer(void *arg) {
    int p = (int)(long)arg;
    for (int k = 1; k &lt;= ITEMS; k++) {
        if ((k - 1) % P != p) continue;              /* this producer's share of items */
        sem_wait(&amp;empty_s);                          /* TODO 1 (completed): wait for a free slot   */
        LOCK();                                      /*   mutex protects in/out indices            */
        int slot = in; sched_yield(); buf[slot] = k; in = (slot + 1) % BUF;
        UNLOCK();
        sem_post(&amp;full_s);                           /*   signal: one more full slot               */
    }
    return NULL;
}
static void *consumer(void *arg) {
    (void)arg;
    for (int n = 0; n &lt; ITEMS / C; n++) {
        sem_wait(&amp;full_s);                           /* TODO 2 (completed): wait for a full slot   */
        LOCK();
        int slot = out; sched_yield(); int item = buf[slot]; out = (slot + 1) % BUF;
        UNLOCK();
        sem_post(&amp;empty_s);                          /*   signal: one more free slot               */
        atomic_fetch_add(&amp;sum, item); atomic_fetch_add(&amp;seen[item], 1);
    }
    return NULL;
}
int main(int argc, char **argv) {
    P = argc &gt; 1 ? atoi(argv[1]) : 1; C = argc &gt; 2 ? atoi(argv[2]) : 1;
    if (argc &gt; 3 &amp;&amp; !strcmp(argv[3], "nomutex")) use_mutex = 0;
    if (ITEMS % P || ITEMS % C) { fprintf(stderr, "P and C must divide %d\n", ITEMS); return 2; }
    sem_init(&amp;empty_s, 0, BUF); sem_init(&amp;full_s, 0, 0);
    pthread_t tp[P], tc[C];
    for (long i = 0; i &lt; P; i++) pthread_create(&amp;tp[i], NULL, producer, (void *)i);
    for (long i = 0; i &lt; C; i++) pthread_create(&amp;tc[i], NULL, consumer, NULL);
    for (int i = 0; i &lt; P; i++) pthread_join(tp[i], NULL);
    for (int i = 0; i &lt; C; i++) pthread_join(tc[i], NULL);
    int once = 1; for (int k = 1; k &lt;= ITEMS; k++) if (seen[k] != 1) once = 0;
    printf("P=%d C=%d mutex=%s  sum=%ld  each item exactly once: %s\n", P, C, use_mutex ? "yes" : "NO", (long)sum,
once ? "yes" : "NO");
    return (sum == 210 &amp;&amp; once) ? 0 : 1;
}
```

Single runs

```
$ ./bb 1 1
P=1 C=1 mutex=yes  sum=210  each item exactly once: yes
$ ./bb 4 4
P=4 C=4 mutex=yes  sum=210  each item exactly once: yes
```

100 runs each, with and without the mutex (run100.sh)

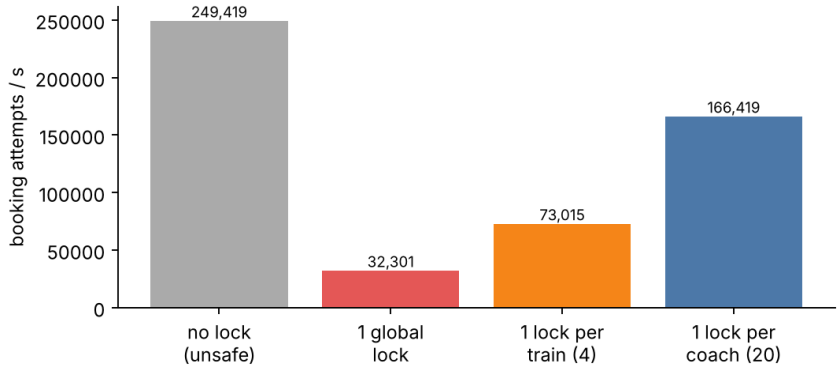
```
args: 1 1  -&gt;  correct runs: 100 / 100   incorrect: 0
args: 4 4  -&gt;  correct runs: 100 / 100   incorrect: 0
args: 1 1 nomutex  -&gt;  correct runs: 100 / 100   incorrect: 0
args: 4 4 nomutex  -&gt;  correct runs: 0 / 100   incorrect: 100
example incorrect run: P=4 C=4 mutex=NO  sum=197  each item exactly once: NO
```

With the mutex, 1×1 and 4×4 were correct in 100 of 100 runs (sum always 210, each item exactly once). **Without the mutex:** with 1 producer and 1 consumer it is still correct (100/100), because only one thread ever changes `in` and only one changes `out`, and the semaphores already order the hand-over. With 4 producers and 4 consumers it was **wrong in 100 of 100 runs** (example: sum 197): two producers read the same `in` and overwrite each other's item, and two consumers read the same `out` and take the same item, so some items are lost and some consumed twice. (To make the race show reliably I put a `sched_yield()` between reading and using the index; without it the race exists but appears less often.)

8. Deployment decision: one lock for the system, per train or per coach

`booking.c`: 4 trains × 5 coaches × 40 seats = 800 seats; 8 agent threads × 400 random booking attempts; a booking checks the seat, then performs a 50 µs payment/database write, then marks it sold. Median of 3 runs; “seats sold twice” counts duplicates.

Strategy	Locks	Time (s)	Attempts / s	Speed-up vs global	Seats sold twice
No lock	0	0.013	249,419	7.72×	[27, 22, 19]
One global lock	1	0.099	32,301	1.00×	[0, 0, 0]
One lock per train	4	0.044	73,015	2.26×	[0, 0, 0]
One lock per coach	20	0.019	166,419	5.15×	[0, 0, 0]



Evaluation: without locks the booking is fastest but sold seats twice (19–27 duplicates per run) – unacceptable. The global lock is correct but only ≈ 32,000 attempts/s because all 8 agents queue behind one another's 50 µs write. A lock per train allows 4 bookings in parallel (2.3×); a lock per coach allows up to 20 (5.1×). All three locking strategies have zero double sales and the same number of sales (781).

Decision: one lock per coach (each coach's seat map has its own mutex). It is correct, gives the highest measured throughput, and bookings for different coaches or trains never wait for each other. Costs: 20 locks instead of 1 (small memory), and any operation that touches several coaches (e.g. booking a family across two coaches, or a train-wide report) must acquire the locks in a fixed order (coach id) to avoid deadlock; those rarer operations can take the per-train lock set in order. Per-coach is not unlimited: the gain is limited by the number of agents and by how many requests target the same coach (popular coaches become hot spots).

9. Alternative concept: atomic operations or a spin lock instead of semaphore + mutex

Solution: for a plain counter, `atomic_fetch_add(&sold, 1)` (C11) performs the read-modify-write as one indivisible hardware instruction (no lock, no waiting queue). A spin lock replaces the sleeping lock by a busy loop: `while (atomic_flag_test_and_set(&l)) ; counter++; atomic_flag_clear(&l);`. Measured with `counter.c` (500,000 updates per thread; time per update = wall time ÷ total updates; CPU time includes all threads):

Method	Threads	ns per update	CPU time (s)	Result
mutex	1	16.9	0.009	correct
mutex	4	49.4	0.156	correct
mutex	16	53.6	0.780	correct

Method	Threads	ns per update	CPU time (s)	Result
sem	1	21.8	0.012	correct
sem	4	197.5	0.781	correct
sem	16	223.9	3.501	correct
spin	1	9.8	0.006	correct
spin	4	80.3	0.314	correct
spin	16	473.7	7.545	correct
atomic	1	6.2	0.004	correct
atomic	4	30.2	0.118	correct
atomic	16	32.2	0.496	correct

Time per update: with 1 thread (no contention) atomic ≈ 6 ns, spin 10 ns, mutex 17 ns, semaphore 22 ns. Under contention (4 and 16 threads on only 2 cores) the atomic counter stays at ≈ 30 ns, the mutex ≈ 50 ns, the semaphore ≈ 200 ns, and the spin lock degrades badly (80 ns with 4, 474 ns with 16 threads). **CPU wasted by spinning:** with 16 threads the spin lock used 7.5 s of CPU for 8 million updates, 15× the atomic counter (0.50 s) and 10× the mutex (0.78 s); a thread that is spinning when the lock holder is pre-empted burns its whole time slice doing nothing useful. All methods updated the counter correctly. Complexity: each update is O(1) for all methods; under contention the expected waiting grows with the number of contenders p, O(p) for lock-based methods.

Long critical section (4 threads, each update holds the lock for a 50 µs wait, 300 updates each):

```
--- long critical section (50 us wait inside, 4 threads x 300 updates)
method threads ns_per_update cpu_seconds correct
mutex 4 129505.0 0.034 ok
sem 4 123307.3 0.033 ok
spin 4 131838.9 0.277 ok
```

Elapsed time per update is the same (≈ 125–130 µs, since the critical sections are serialised anyway) but the spin lock used **0.277 s of CPU against 0.033 s for the mutex and 0.034 s for the semaphore** (≈ 8×): spinners burn CPU while the holder is waiting. **So spin locks suit only very short critical sections (a few instructions) on multicore machines, atomics suit single-variable updates, and mutexes/semaphores (sleeping locks) suit long critical sections** such as a booking with a payment or database write.

10. Final recommendation

Use **one mutex per coach** to protect each seat map (measured 5.1× the throughput of one global lock with no double sales), a **counting semaphore or atomic counter** for available-seat counts, a monitor (mutex + condition variables) wherever agents must wait for a condition such as “a seat becomes free”, and fixed lock ordering when an operation needs several locks. Do not use Peterson's algorithm or spin locks on production booking paths.

Limitations

Timings were taken on a 2-core virtualised container and the booking test has only 3 short repetitions (consistent within ≈ 10%); absolute numbers will differ on your machine. The 50 µs payment delay is simulated. Peterson's failure depends on the CPU's memory model (shown here on x86). Run the programs yourself, commit `Task05/*.c, run100.sh` and outputs, and add your name, register number and GitHub link.